

ui.switch 全面详解（基于 NiceGUI 文档）

ui.switch 是 NiceGUI 中用于实现布尔值开关控制的核心交互组件，基于 Quasar 的 QSwitch 组件实现。它以“滑动开关”的视觉形式呈现，支持启用 / 禁用状态切换、状态监听、数据绑定和样式定制，适用于功能开关、权限控制、参数配置等需要直观二值选择的场景。其核心优势在于交互直观、样式紧凑，相比 ui.checkbox 更侧重“功能开关”的视觉表达，相比 ui.toggle 更简洁轻量。以下从核心特性、基础用法、高级功能等维度展开全面解析。

一、核心基础

1. 组件本质与核心特性

- 底层依赖：基于 Quasar 的 QSwitch 组件，继承其成熟的滑动交互逻辑、样式体系和无障碍支持。
- 核心功能：仅支持布尔值状态（开启 `True` / 关闭 `False`），通过点击开关滑块或标签触发状态切换，支持初始状态配置、状态变化监听和禁用状态。
- 交互特性：点击开关滑块或其关联的文本标签均可切换状态，滑块滑动时有平滑过渡动画，视觉反馈清晰。

2. 初始化参数（核心配置）

参数名	类型	说明	默认值
text	str	开关右侧的文本标签（描述开关功能）	""
value	bool	初始状态（ <code>True</code> 为开启， <code>False</code> 为关闭）	False
on_change	Callable	状态变化时触发的回调函数（ <code>e.value</code> 为当前状态）	-
color	str / None	开启状态的滑块颜色（支持 Quasar 颜色、Tailwind 颜色、CSS 颜色）	"primary"
disabled	bool	是否禁用组件（禁用后不可交互，视觉灰度）	False
label_position	str	文本标签位置（可选值："left"、"right"）	"right"
dense	bool	是否使用紧凑模式（减少组件高度）	False

二、基础使用示例

1. 最简用法（文本标签 + 状态联动）

展示基础文本标签、初始状态配置，以及与其他组件的状态联动（如控制元素显示 / 隐藏）：

```
from nicegui import ui

# 基础开关（带文本标签，初始关闭）
switch = ui.switch('启用深色模式', value=False)

# 联动页面背景色：开启时切换为深色，关闭时切换为浅色
ui.query('body').bind_style(
    'background-color',
```

```

        switch, 'value',
        lambda x: '#2d3748' if x else '#ffffff'
    )
ui.query('body').bind_style(
    'color',
    switch, 'value',
    lambda x: '#ffffff' if x else '#000000'
)

ui.run()

```

效果：

- 开关右侧显示文本“启用深色模式”，初始为关闭状态（浅色背景、黑色文字）；
- 点击开关切换为开启状态，页面背景色变为深色，文字变为白色，实现实时联动。

2. 状态变化监听（用户交互 + 手动修改）

区分“用户点击触发”和“代码手动修改”两种状态变化场景，展示 `on_change` 事件的用法：

```

from nicegui import ui

with ui.column().classes('gap-4'):
    # 开关组件（初始关闭）
    switch = ui.switch('功能开关', on_change=lambda e: ui.notify(f'功能已{"开启" if e.value else "关闭"}'))

    # 手动修改开关状态的按钮
    ui.button('切换开关状态', on_click=lambda: switch.set_value(not switch.value))

ui.run()

```

效果：

- 点击开关或按钮均可切换状态，且都会触发 `on_change` 回调，弹出对应的通知；
- 开关状态变化时，滑块有平滑滑动动画，视觉反馈清晰。

三、核心功能与进阶用法

1. 数据绑定（双向同步）

通过 `bind_value` 实现开关与数据对象的双向绑定，数据变化时组件自动更新，反之亦然：

```

from nicegui import ui

class AppConfig:
    def __init__(self):
        self.notifications_enabled = True # 初始开启通知
        self.auto_save_enabled = False   # 初始关闭自动保存

config = AppConfig()

```

```

with ui.column().classes('gap-4'):
    # 开关与数据对象绑定（通知开关）
    ui.switch('启用通知').bind_value(config, 'notifications_enabled')

    # 开关与数据对象绑定（自动保存开关）
    ui.switch('自动保存', color='green').bind_value(config, 'auto_save_enabled')

    # 显示绑定数据的实时状态
    ui.label().bind_text_from(
        config, 'notifications_enabled',
        lambda x: f'通知状态: {"开启" if x else "关闭"}'
    )
    ui.label().bind_text_from(
        config, 'auto_save_enabled',
        lambda x: f'自动保存: {"开启" if x else "关闭"}'
    )

    # 手动修改数据对象，组件自动同步
    ui.button('一键开启所有功能', on_click=lambda: (
        setattr(config, 'notifications_enabled', True),
        setattr(config, 'auto_save_enabled', True)
    )).classes('mt-2')

ui.run()

```

效果：

- 点击开关切换状态，数据对象 `config` 对应的属性自动同步；
- 点击“一键开启所有功能”按钮修改数据对象，两个开关自动切换为开启状态，实现双向绑定同步。

2. 样式定制（颜色、尺寸、标签位置）

通过 `props`、`classes` 和 `style` 定制开关的颜色、尺寸、标签位置等样式：

```

from nicegui import ui

with ui.column().classes('gap-4'):
    # 1. 自定义开启颜色（红色）+ 初始开启
    ui.switch('紧急模式', value=True, color='red')

    # 2. 紧凑模式+大尺寸
    ui.switch('紧凑+大尺寸', dense=True).props('size=lg')

    # 3. 标签在左侧（默认在右侧）
    ui.switch('标签在左', label_position='left')

    # 4. 自定义滑块颜色和轨道颜色（通过 CSS）
    ui.switch('自定义样式', classes='mt-2').style(''
        --q-switch-color: #667eea; /* 开启时滑块颜色 */
        --q-switch-track-color: #e2e8f0; /* 关闭时轨道颜色 */
        --q-switch-checked-track-color: #c5ceeb; /* 开启时轨道颜色 */
    )

```

```
'''  
  
ui.run()
```

效果：

- 四个开关分别展示不同样式，支持开启颜色、尺寸、标签位置、滑块 / 轨道颜色的定制，适配不同界面设计需求。

3. 禁用状态与条件启用

通过 `set_disabled` 禁用开关，或实现基于前置条件的开关启用逻辑（如勾选协议后才允许开启功能）：

```
from nicegui import ui  
  
# 前置条件：同意协议复选框  
agree_switch = ui.switch('我已阅读并同意协议', value=False)  
  
# 依赖前置条件的功能开关（初始禁用）  
feature_switch = ui.switch('启用高级功能', disabled=True, color='purple')  
  
# 前置条件变化时，启用/禁用功能开关  
def on_agree_change(e):  
    feature_switch.set_disabled(not e.value)  
    # 取消协议时，自动关闭功能  
    if not e.value:  
        feature_switch.set_value(False)  
  
agree_switch.on_change(on_agree_change)  
  
# 禁用开关的按钮  
ui.button('禁用功能开关', on_click=lambda: feature_switch.set_disabled(True)).classes('mt-4')  
  
ui.run()
```

效果：

- 未开启“我已阅读并同意协议”时，“启用高级功能”开关禁用，无法操作；
- 开启协议开关后，功能开关启用，可正常切换状态；
- 取消协议开关时，功能开关自动关闭并禁用，确保逻辑一致性。

4. 开关组联动（互斥开关）

实现多个开关的互斥逻辑（如“仅允许开启一个功能”）：

```
from nicegui import ui  
  
# 互斥开关组  
switches = []  
  
def on_switch_change(active_switch):  
    # 关闭其他所有开关
```

```
for switch in switches:
    if switch != active_switch and switch.value:
        switch.set_value(False)

# 创建三个互斥开关
with ui.column().classes('gap-2'):
    for name, color in [('功能A', 'blue'), ('功能B', 'green'), ('功能C', 'orange')]:
        switch = ui.switch(name, color=color)
        switch.on_change(lambda e, s=switch: on_switch_change(s))
        switches.append(switch)

ui.run()
```

效果：

- 点击任意开关开启时，其他已开启的开关自动关闭，确保同一时间仅一个功能处于开启状态。

四、核心属性与方法

1. 常用属性

属性名	类型	说明
classes	Classes[Self]	CSS 类（支持 Tailwind/Quasar 样式）
disabled	BindableProperty	是否禁用（可绑定数据动态控制，禁用后不可交互）
html_id	str	HTML 元素 ID（2.16.0+ 版本支持）
text	BindableProperty	文本标签（可绑定数据动态修改）
value	BindableProperty	开启状态（True / False，可绑定数据）
visible	BindableProperty	是否可见（可绑定数据）
color	str	开启状态的滑块颜色（支持 Quasar/Tailwind/CSS 颜色）
label_position	str	标签位置（"left"/"right"）
dense	bool	是否紧凑模式

2. 关键方法

(1) 状态控制

- `disable()`：禁用组件（不可交互，视觉灰度）
- `enable()`：启用组件
- `set_disabled(value: bool)`：设置启用状态（True 禁用，False 启用）
- `set_visibility(visible: bool)`：设置组件可见性

(2) 属性修改

- `set_value(value: bool)`: 动态修改开启状态 (`True` 开启, `False` 关闭, 触发 `on_change` 事件)
- `set_text(text: str)`: 动态修改文本标签
- `set_color(color: str)`: 动态修改开启状态的滑块颜色
- `update()`: 修改属性后, 调用此方法刷新界面 (绑定数据时无需手动调用)

(3) 事件与绑定

- `on_change(callback)`: 绑定状态变化事件 (`e.value` 为当前状态, 覆盖所有触发方式)
- `on(type: str, handler)`: 订阅任意 DOM 事件 (如 `click` 仅监听用户点击, `mousedown` 等)
- `bind_value(target_object, target_name)`: 双向绑定开启状态到目标对象的属性
- `bind_text_from(target_object, target_name)`: 单向绑定文本标签从目标对象
- `bind_disabled_from(target_object, target_name)`: 单向绑定禁用状态从目标对象

(4) 其他实用方法

- `tooltip(text: str)`: 为组件添加悬停提示
- `delete()`: 彻底删除组件
- `mark(*markers)`: 添加标记 (用于测试或元素查询)

五、使用场景与注意事项

1. 适用场景

- 功能开关: 如深色模式、通知开启 / 关闭、自动保存等功能的启用控制。
- 权限控制: 如基于用户角色的功能权限开关 (管理员可见 / 可操作)。
- 参数配置: 如系统设置中的各类二值配置项 (如是否允许匿名访问、是否自动更新)。
- 状态切换: 如任务状态 (激活 / 暂停)、设备状态 (开启 / 关闭) 等的直观控制。

2. 关键注意事项

- 状态类型限制: 组件 `value` 仅支持布尔值 (`True/False`), 不支持其他类型 (如字符串、数字)。
- 颜色优先级: `color` 参数仅控制开启状态的滑块颜色, 关闭状态的颜色由主题默认样式控制, 可通过 CSS 变量自定义 (如 `--q-switch-track-color`)。
- 标签交互: 文本标签与开关联动, 点击标签即可切换状态, 无需单独绑定事件。
- 版本兼容性: `html_id` 属性仅在 2.16.0+ 版本支持, `bind_disabled` 的 `strict` 参数在 3.0.0+ 版本支持, 使用时需确认版本。
- 与其他组件的区别:
 - 需直观功能开关, 样式紧凑 → 优先使用 `ui.switch`;
 - 需多选项互斥选择 → 优先使用 `ui.toggle` 或 `ui.radio`;
 - 需表单勾选或多选列表 → 优先使用 `ui.checkbox`。

六、进阶示例：带状态记忆的开关组（结合本地存储）

实现开关状态的本地存储（页面刷新后保留状态），适用于系统设置类场景：

```
from nicegui import ui
import json
from pathlib import Path

# 本地存储文件路径
STORAGE_FILE = Path('switch_states.json')

# 加载本地存储的状态
def load_states():
    if STORAGE_FILE.exists():
        with open(STORAGE_FILE, 'r') as f:
            return json.load(f)
    return {'dark_mode': False, 'notifications': True, 'auto_save': False}

# 保存状态到本地
def save_state(key, value):
    states = load_states()
    states[key] = value
    with open(STORAGE_FILE, 'w') as f:
        json.dump(states, f, indent=2)

# 初始加载状态
initial_states = load_states()

with ui.column().classes('gap-4'):
    # 深色模式开关（绑定本地存储状态）
    dark_switch = ui.switch('深色模式', value=initial_states['dark_mode'], color='black')
    dark_switch.on_change(lambda e: save_state('dark_mode', e.value))

    # 联动页面背景色
    ui.query('body').bind_style(
        'background-color',
        dark_switch, 'value',
        lambda x: '#2d3748' if x else 'ffffff'
    )

    # 通知开关（绑定本地存储状态）
    notify_switch = ui.switch('启用通知', value=initial_states['notifications'],
    color='blue')
    notify_switch.on_change(lambda e: save_state('notifications', e.value))

    # 自动保存开关（绑定本地存储状态）
    save_switch = ui.switch('自动保存', value=initial_states['auto_save'], color='green')
    save_switch.on_change(lambda e: save_state('auto_save', e.value))

ui.run()
```

效果：

- 开关初始状态加载自本地 `switch_states.json` 文件；

- 切换开关状态时，自动保存到本地文件；
- 页面刷新后，开关状态从本地文件加载，保留之前的配置，实现状态记忆功能。